

API REFERENCE

MonCha Backend API

All 19 endpoints: method, URL, CRUD, payload, response and errors. Includes register, login and logout.

Base URL	<code>https://moncha-backend.vinothjv4-tech.workers.dev</code>
Version	Deployed version a67dc3fb
Format	JSON over HTTPS
Auth	Bearer token (register or login)
Tested	All endpoints tested on the live backend
Date	3 October 2026

Every endpoint served by the backend (19 in total). All examples below are real responses captured from the live deployment on 2026-10-03 (50 of 50 full test cases, 26 of 26 re-checks after the audit changes, and 22 of 22 auth cases passed, see [Test report](#)).

- **Base URL (deployed):** `https://moncha-backend.vinothjv4-tech.workers.dev`
- **Base URL (local):** `http://localhost:4000` (`pnpm --filter @moncha/worker api`), see [Local API differences](#)
- **Database:** Neon `dev` branch
- **Format:** JSON in, JSON out (`content-type: application/json; charset=utf-8`)
- **Postman:** `apps/worker/postman/moncha-worker-api.postman_collection.json`

Contents

- [Summary of all endpoints](#)
 - [Common rules](#) (headers, errors, status codes, CORS)
 - [Service:](#) `GET /` , `GET /health`
 - [Schedules:](#) list, create, delete one, delete by country, run history
 - [Discovery and jobs:](#) start a country crawl, get job status
 - [Leads:](#) list, counts, get one, create, re-audit
 - [Auth:](#) register, login, logout, current user
 - [How a lead's queue is decided](#)
 - [Enums](#)
 - [Background processing](#)
 - [Local API differences](#)
 - [Test report](#)
-

Summary of all endpoints

#	Method	Endpoint	CRUD	What it does	Success	Tables
1	GET	/	Read	Service index	200	none
2	GET	/health	Read	Health and database check	200	none (SELECT 1)
3	OPTIONS	any path	n/a	CORS preflight	204	none
4	GET	/api/v1/schedules	Read	List daily schedules, grouped by country	200	DiscoverySchedule
5	POST	/api/v1/schedules	Create	Add a daily run time for a country	201	DiscoverySchedule
6	DELETE	/api/v1/schedules/:id	Delete	Remove one schedule time	200	DiscoverySchedule
7	DELETE	/api/v1/schedules?country=	Delete	Remove all schedule times of a country	200	DiscoverySchedule
8	GET	/api/v1/schedules/runs	Read	Country crawl run history	200	JobRun
9	POST	/api/v1/discovery/country	Create	Queue a country crawl now	202	JobRun
10	GET	/api/v1/jobs/:id	Read	Status of any background job	200	JobRun
11	GET	/api/v1/leads	Read	Paginated lead list with filters	200	Lead, Company, Website
12	GET	/api/v1/leads/counts	Read	Number of leads per queue	200	Lead
13	GET	/api/v1/leads/:id	Read	One lead with company and website	200	Lead, Company, Website
14	POST	/api/v1/leads	Create (or Update if duplicate)	Add a lead manually	201 / 200	Company, SourceRecord, Lead, Website, JobRun
15	POST	/api/v1/leads/:id/audit	Create + Update	Queue a website re-audit for a lead	202	Lead, Website, JobRun
16	POST	/api/v1/auth/register	Create	Create an account and sign in	201	User, Session
17	POST	/api/v1/auth/login	Create	Sign in with email and password, get a token	200	User (read), Session

#	Method	Endpoint	CRUD	What it does	Success	Tables
18	POST	/api/v1/auth/logout	Update	Sign out (revoke the token)	200	Session
19	GET	/api/v1/auth/me	Read	The signed-in user	200	Session, User

There are no PUT or PATCH endpoints. To change a schedule time, delete the old one (#6) and create the new one (#5).

Common rules

Request headers

Header	Required	Description
content-type: application/json	For POST with a body	Body must be valid JSON. An empty body is treated as <code>{}</code> .
authorization: Bearer <token>	Only for /api/v1/auth/logout and /api/v1/auth/me	Token from register or login. Other endpoints do not check it yet.
x-tenant-id	No	Tenant to read and write. Default: <code>tenant_moncha_internal (DEFAULT_TENANT_ID)</code> .
x-api-key	Only if <code>WORKER_API_KEY</code> is set on the server	Currently not set on the deployed backend, so it is not needed.

Error format

Every error has the same shape:

```
{
  "error": {
    "code": "validation_error",
    "message": "Invalid request",
    "details": { "formErrors": [], "fieldErrors": { "time": ["Use 24-hour HH:mm"] } }
  }
}
```

`details` is only present for field validation errors.

Status	code	When
400	validation_error	Missing or invalid field, invalid JSON body, unsupported country, unknown time zone
401	unauthorized	Wrong email or password; missing, expired or logged-out token; or wrong <code>x-api-key</code> (only when <code>WORKER_API_KEY</code> is set)
404	not_found	Unknown route, or the schedule, job or lead does not exist

Status	code	When
405	method_not_allowed	Local API only: known path, wrong method (deployed returns 404)
409	conflict	Schedule already exists for that country and time, or an account with that email already exists
413	payload_too_large	Local API only: body larger than 1 MB
500	internal_error	Unexpected server error
503	configuration_error	DATABASE_URL is not set on the server

Unknown routes (and unsupported methods on known paths) return:

```
{ "error": { "code": "not_found", "message": "No route for GET /api/v1/nope" } }
```

CORS

All responses include CORS headers. The request `origin` is echoed back, so any frontend origin works.

```
Access-Control-Allow-Origin: <request origin or *>
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD
Access-Control-Allow-Headers: <requested headers or Content-Type, Authorization, x-tenant-id, x-api-key>
Access-Control-Max-Age: 86400
```

Supported countries

Singapore (SG), Malaysia (MY), Japan (JP). Anywhere a `country` is accepted you can send the name or the code, for example `"Singapore"` or `"SG"`.

Dates

All timestamps are ISO 8601 in UTC, for example `2026-10-03T14:00:00.000Z`.

Service

1. GET /

CRUD: Read. Service index.

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/
```

200 OK

```
{
  "service": "moncha-backend",
  "status": "ok",
  "endpoints": ["/health", "/api/v1/auth/login", "/api/v1/schedules", "/api/v1/schedules/runs",
"/api/v1/leads"]
}
```

The `endpoints` list is informational only; this document is the full list.

2. GET /health

CRUD: Read. Checks the backend and database connection.

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/health
```

200 OK (database reachable)

```
{
  "ok": true,
  "db": "up",
  "tenantId": "tenant_moncha_internal",
  "countries": ["Singapore (SG)", "Malaysia (MY)", "Japan (JP)"]
}
```

503 when the database is down: `"ok": false`, `"db": "down: <reason>"`.

Field	Type	Description
<code>ok</code>	boolean	<code>true</code> when the database answered
<code>db</code>	string	<code>"up"</code> or <code>"down: <reason>"</code>
<code>tenantId</code>	string	Tenant used for this request
<code>countries</code>	string[]	Countries the crawler supports

Schedules

A schedule is one daily run time for one country. A country can have several times per day. When a time is due, the background worker starts a country crawl (see [Background processing](#)).

Schedule object

Field	Type	Description
id	string	Schedule id
countryCode	string	SG , MY or JP
country	string	Country name
time	string	Local time HH:mm (24-hour)
timezone	string	IANA time zone, for example Asia/Kuala_Lumpur
nextRunAt	string	Next run in UTC
nextRunDay	string	"today" , "tomorrow" or YYYY-MM-DD in the schedule's time zone
lastRunAt	string null	Last time it started a run

4. GET /api/v1/schedules

CRUD: Read. All schedules, grouped by country (sorted by country name, then time).

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/schedules
```

200 OK

```

{
  "countries": [
    {
      "countryCode": "JP",
      "country": "Japan",
      "timesPerDay": 2,
      "schedules": [
        {
          "id": "sched_9ff62537650f48b5",
          "countryCode": "JP",
          "country": "Japan",
          "time": "09:15",
          "timezone": "Asia/Tokyo",
          "nextRunAt": "2026-10-04T00:15:00.000Z",
          "nextRunDay": "tomorrow",
          "lastRunAt": null
        },
        {
          "id": "sched_908b52bfda9d4422",
          "countryCode": "JP",
          "country": "Japan",
          "time": "21:45",
          "timezone": "Asia/Tokyo",
          "nextRunAt": "2026-10-03T12:45:00.000Z",
          "nextRunDay": "today",
          "lastRunAt": null
        }
      ]
    },
    {
      "countryCode": "SG",
      "country": "Singapore",
      "timesPerDay": 1,
      "schedules": [
        {
          "id": "sched_86698cf497e64b0c",
          "countryCode": "SG",
          "country": "Singapore",
          "time": "22:00",
          "timezone": "Asia/Kuala_Lumpur",
          "nextRunAt": "2026-10-03T14:00:00.000Z",
          "nextRunDay": "today",
          "lastRunAt": null
        }
      ]
    }
  ]
}

```

With no schedules: { "countries": [] }.

5. POST /api/v1/schedules

CRUD: Create. Adds one daily run time.

Body

Field	Type	Required	Rules
country	string	Yes	Supported country name or code
time	string	Yes	24-hour HH:mm , 00:00 to 23:59 (9:00 is rejected, use 09:00)
timezone	string	No	IANA zone. Default Asia/Kuala_Lumpur (same clock as Singapore, UTC+8)

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/schedules \
-H "content-type: application/json" \
-d '{ "country": "Singapore", "time": "22:00" }'
```

201 Created (a schedule object)

```
{
  "id": "sched_86698cf497e64b0c",
  "countryCode": "SG",
  "country": "Singapore",
  "time": "22:00",
  "timezone": "Asia/Kuala_Lumpur",
  "nextRunAt": "2026-10-03T14:00:00.000Z",
  "nextRunDay": "today",
  "lastRunAt": null
}
```

Errors

Status	Example message
400	fieldErrors.time: ["Use 24-hour HH:mm"]
400	Unsupported country "France". Supported: Singapore (SG), Malaysia (MY), Japan (JP)
400	Unknown time zone "Mars/Base"
400	Invalid JSON body
409	Singapore is already scheduled at 22:00

6. DELETE /api/v1/schedules/:id

CRUD: Delete. Removes one schedule time.

```
curl -X DELETE https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/schedules/sched_86698cf497e64b0c
```

200 OK

```
{ "deleted": 1 }
```

404 { "error": { "code": "not_found", "message": "Schedule not found" } }

7. DELETE /api/v1/schedules?country=<country>

CRUD: Delete. Removes every schedule time of one country.

Query	Required	Description
country	Yes	Supported country name or code

```
curl -X DELETE "https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/schedules?country=Japan"
```

200 OK (`deleted` is the number removed, can be `0`)

```
{ "deleted": 2 }
```

400 when `country` is missing (`fieldErrors.country`) or unsupported.

8. GET /api/v1/schedules/runs

CRUD: Read. History of country crawls (scheduled and manual), newest first.

Query	Required	Default	Rules
limit	No	50	Integer 1 to 200

```
curl "https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/schedules/runs?limit=2"
```

200 OK

```

{
  "runs": [
    {
      "id": "job_f77ffd5d69b4467b",
      "day": "2026-10-03",
      "countryCode": "SG",
      "country": "Singapore",
      "time": "13:11",
      "timezone": "Asia/Kuala_Lumpur",
      "trigger": "manual",
      "status": "pending",
      "found": 0,
      "saved": 0,
      "skipped": 0,
      "stoppedReason": null,
      "error": null,
      "startedAt": null,
      "finishedAt": null
    },
    {
      "id": "cmumkan1n0003g5z8wmghre8k",
      "day": "2026-09-29",
      "countryCode": "SG",
      "country": "Singapore",
      "time": "18:57",
      "timezone": "Asia/Kuala_Lumpur",
      "trigger": "schedule",
      "status": "done",
      "found": 60,
      "saved": 57,
      "skipped": 3,
      "stoppedReason": "run_budget_reached",
      "error": null,
      "startedAt": "2026-09-29T10:57:05.060Z",
      "finishedAt": "2026-09-29T10:57:44.906Z"
    }
  ]
}

```

Field	Type	Description
<code>id</code>	string	Job id (use with <code>GET /api/v1/jobs/:id</code>)
<code>day</code>	string	Local date of the run, <code>YYYY-MM-DD</code>
<code>time</code>	string	Local time of the run, <code>HH:mm</code>
<code>trigger</code>	string	<code>schedule</code> (from a schedule) or <code>manual</code> (from <code>POST /api/v1/discovery/country</code>)
<code>status</code>	string	<code>pending</code> , <code>running</code> , <code>done</code> , <code>failed</code>
<code>found</code>	number	Places found
<code>saved</code>	number	New leads saved
<code>skipped</code>	number	Already in the database

Field	Type	Description
<code>stoppedReason</code>	string null	Why the run stopped, for example <code>run_budget_reached</code>
<code>error</code>	string null	Last error message
<code>startedAt</code> , <code>finishedAt</code>	string null	UTC timestamps

400 when `limit` is outside 1 to 200.

Discovery and jobs

9. POST /api/v1/discovery/country

CRUD: Create. Queues a country-wide crawl (Google Places across the country's cities and industries). Returns immediately; the background worker runs it.

Body

Field	Type	Required	Default	Rules
country	string	Yes		Supported country name or code
source	string	No	google_places	Only google_places
cities	string[]	No	All search areas of the country	At least 1 item if sent
industries	string[]	No	Built-in industry list	At least 1 item if sent
maxPages	number	No	3	Integer 1 to 3 (result pages per search)
maxSearches	number	No	Server budget	Integer ≥ 1
maxCallsPerDay	number	No	Server budget (500)	Integer ≥ 1
reset	boolean	No	false	Restart the country crawl from the beginning

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/discovery/country \
-H "content-type: application/json" \
-d '{ "country": "Singapore", "cities": ["Singapore"], "industries": ["dental clinic"], "maxPages": 1 }'
```

202 Accepted

```
{ "id": "job_453a40d1336341e3", "status": "pending", "type": "country_discovery" }
```

400 when `country` is missing or unsupported, or `maxPages` is above 3.

10. GET /api/v1/jobs/:id

CRUD: Read. Status of any background job (country crawl or website audit).

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/jobs/job_453a40d1336341e3
```

200 OK

```
{
  "id": "job_453a40d1336341e3",
  "tenantId": "tenant_moncha_internal",
  "type": "country_discovery",
  "status": "pending",
  "payload": {
    "mode": "country",
    "reset": false,
    "cities": ["Singapore"],
    "origin": "manual",
    "country": "Singapore",
    "maxPages": 1,
    "industries": ["dental clinic"],
    "countryCode": "SG",
    "maxSearches": 1,
    "maxCallsPerDay": 1
  },
  "result": null,
  "attempts": 0,
  "maxAttempts": 1,
  "runAfter": "2026-10-03T05:13:16.824Z",
  "lockedAt": null,
  "lockedBy": null,
  "lastError": null,
  "dedupeKey": null,
  "startedAt": null,
  "finishedAt": null,
  "createdAt": "2026-10-03T05:13:16.952Z"
}
```

Field	Type	Description
type	string	country_discovery, website_audit, places_discovery, csv_import
status	string	pending → running → done or failed
payload	object	Job input. Country crawl: as above. Website audit: { leadId, companyId, url, force }
result	object null	Job output when finished. Country crawl: { found, created, duplicates, stoppedReason, calls, ... }
attempts / maxAttempts	number	Tries so far / allowed
runAfter	string	Earliest time the worker may pick it up
lockedAt / lockedBy	string null	Set while a worker holds the job
lastError	string null	Last failure message
dedupeKey	string null	Prevents duplicate jobs

404 { "error": { "code": "not_found", "message": "Job not found" } }

Leads

Lead object (list items and `GET /api/v1/leads/:id`)

```
{
  "id": "cmugv7jqx000hg5b0ptdja5hp",
  "tenantId": "tenant_moncha_internal",
  "companyId": "cmugv7fk4000dg5b0vdcbsmj",
  "queue": "QUALIFIED",
  "assistantVerdict": "NO_ASSISTANT",
  "assistantVendor": null,
  "qualificationReason": "no_assistant_high_confidence",
  "latestAuditId": "cmukorq5p00fsg5b8qkda1b5q",
  "version": 3,
  "createdAt": "2026-09-25T11:15:59.289Z",
  "updatedAt": "2026-09-28T03:26:56.210Z",
  "company": {
    "id": "cmugv7fk4000dg5b0vdcbsmj",
    "tenantId": "tenant_moncha_internal",
    "name": "Straits Dental Group Orchard",
    "domain": "straitsdental.com",
    "country": "Singapore",
    "city": "Singapore",
    "phone": "+65 6235 4686",
    "address": "360 Orchard Rd, #04-01 International Building, Singapore 238869",
    "website": {
      "id": "cmugv7n1p000jg5b0lbeqapq3",
      "tenantId": "tenant_moncha_internal",
      "companyId": "cmugv7fk4000dg5b0vdcbsmj",
      "url": "https://straitsdental.com",
      "canonicalUrl": null,
      "status": "ACTIVE",
      "language": null,
      "finalUrl": null,
      "httpStatus": null,
      "title": "Home | Straits Dental | Orchard | Tai Seng | Raffles Place",
      "latestAuditId": null,
      "lastCheckedAt": null
    },
  },
  "sourceRecords": []
}
```

Field	Type	Description
queue	string	Lead queue, see Enums . <code>QUALIFIED</code> means no chatbot found: a sales lead
assistantVerdict	string null	Audit result; <code>null</code> until the first audit finishes
assistantVendor	string null	Chatbot vendor if one was found
qualificationReason	string null	Why it is in this queue, for example <code>no_assistant_high_confidence</code> , <code>no_website</code> , <code>pending_audit</code>

Field	Type	Description
latestAuditId	string null	Last website audit
version	number	Increases on every audit update
company.website	object null	null when the company has no website

On the deployed backend, `company.website` only fills `id`, `url`, `status` and `title`; the other website fields are `null`, and `sourceRecords` is always `[]`.

11. GET /api/v1/leads

CRUD: Read. Paginated lead list, newest first.

Query	Required	Default	Rules
queue	No	QUALIFIED	One of the lead queues
page	No	1	Integer ≥ 1
pageSize	No	25	Integer 1 to 100
search	No		Matches company name or domain (contains, case-insensitive)
country	No		Exact company country, for example <code>Singapore</code>

```
curl "https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/leads?
queue=QUALIFIED&page=1&pageSize=25&search=dental&country=Singapore"
```

200 OK

```
{
  "items": [ { "...": "lead object" } ],
  "total": 100,
  "page": 1,
  "pageSize": 25,
  "totalPages": 4
}
```

`totalPages` is `0` when `total` is `0`.

400 for an invalid `queue`, `page` below 1, or `pageSize` above 100:

```
{
  "error": {
    "code": "validation_error",
    "message": "Invalid request",
    "details": {
      "formErrors": [],
      "fieldErrors": {
        "queue": ["Invalid option: expected one of
\"PENDING_AUDIT\"|\"QUALIFIED\"|\"HAS_ASSISTANT\"|\"NO_WEBSITE\"|\"NEEDS_REVIEW\"|\"INACTIVE\""]
      }
    }
  }
}
```

12. GET /api/v1/leads/counts

CRUD: Read. Number of leads in each queue (for tabs and badges).

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/leads/counts
```

200 OK

```
{
  "PENDING_AUDIT": 0,
  "QUALIFIED": 128,
  "HAS_ASSISTANT": 7,
  "NO_WEBSITE": 76,
  "NEEDS_REVIEW": 0,
  "INACTIVE": 14,
  "openReviewTasks": 0
}
```

All six queues are always present, `0` when empty.

On the deployed backend `openReviewTasks` is always `0`.

13. GET /api/v1/leads/:id

CRUD: Read. One lead with its company and website (lead object above).

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/leads/cmugv7jqx000hg5b0ptdja5hp
```

200 OK → lead object. **404** { "error": { "code": "not_found", "message": "Lead not found" } }

14. POST /api/v1/leads

CRUD: Create (or Update when the domain already exists). Adds a lead by hand.

Body

Field	Type	Required	Description
name	string	Yes	Company name
domain	string	No	Domain or full URL; normalized, so <code>https://www.example.org/</code> becomes <code>example.org</code>
country	string	No	
city	string	No	
phone	string	No	
address	string	No	

What happens

- **New domain:** creates Company, Lead (`PENDING_AUDIT`) and Website, and queues a `website_audit` job. Returns **201** with `auditEnqueued: true` .
- **No domain:** creates Company and Lead (`NO_WEBSITE` , verdict `NOT_APPLICABLE`). No audit. Returns **201** .
- **Domain already exists:** does not create a new lead; fills empty company fields (country, city, phone, address) and returns the existing lead with **200** and `duplicate: true` .

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/leads \
-H "content-type: application/json" \
-d '{ "name": "API Docs Test Web Co", "domain": "example.org", "country": "Singapore", "city": "Singapore" }'
```

201 Created (new, with website)

```
{
  "company": {
    "id": "comp_8a7352a728984f91",
    "tenantId": "tenant_moncha_internal",
    "name": "API Docs Test Web Co",
    "domain": "example.org",
    "country": "Singapore",
    "city": "Singapore",
    "phone": null,
    "address": null
  },
  "lead": {
    "id": "lead_25f6709a53cb4280",
    "tenantId": "tenant_moncha_internal",
    "companyId": "comp_8a7352a728984f91",
    "queue": "PENDING_AUDIT",
    "assistantVerdict": null,
    "assistantVendor": null,
    "qualificationReason": "pending_audit",
    "latestAuditId": null,
    "version": 1,
    "createdAt": "2026-10-03T05:13:25.582Z",
    "updatedAt": "2026-10-03T05:13:25.582Z"
  },
  "duplicate": false,
  "auditEnqueued": true
}
```

201 Created (new, no website): same shape with `"domain": null`, `"queue": "NO_WEBSITE"`, `"assistantVerdict": "NOT_APPLICABLE"`, `"qualificationReason": "no_website"`, `"auditEnqueued": false`.

200 OK (duplicate): same shape with the existing company and lead, `"duplicate": true`, `"auditEnqueued": false`.

400 when `name` is missing:

```
{
  "error": {
    "code": "validation_error",
    "message": "Invalid request",
    "details": { "formErrors": [], "fieldErrors": { "name": ["Invalid input: expected string, received undefined"] } }
  }
}
```

15. POST /api/v1/leads/:id/audit

CRUD: Create + Update. Queues a forced re-audit of the lead's website (sets the website status back to `UNCHECKED` and creates a `website_audit` job). No body.

```
curl -X POST https://moncha-backend.vinothjv4-
tech.workers.dev/api/v1/leads/cmugv7jqx000hg5b0ptdja5hp/audit
```

202 Accepted (new job)

```
{ "id": "job_598f2ccec3cf4711", "status": "pending" }
```

202 Accepted (an audit for this lead is already pending or running; no new job)

```
{ "id": "job_598f2ccec3cf4711", "status": "pending", "deduped": true }
```

404 { "error": { "code": "not_found", "message": "Lead or its website domain not found" } } when the lead does not exist or has no domain.

Auth

Email and password accounts with bearer-token sessions.

How it works

1. Call **register** (new account) or **login** (existing account). Both return a `token`.
2. Store the token on the client and send it as `Authorization: Bearer <token>` on later requests.
3. Call **me** to restore the signed-in user (for example on page reload). A `401` means the token expired or was logged out: send the user to the login page.
4. Call **logout** to end the session. The token stops working immediately on the server.

Rules

- A token is valid for **7 days** (`expiresAt`). Every login creates a new, separate session, so a user can be signed in on several devices; logout ends only the session of the token sent.
- Accounts belong to a tenant (`x-tenant-id` , default `tenant_moncha_internal`). The same email can exist in different tenants. Emails are stored in lower case and matched without regard to case.
- Register always creates the role `operator` . `admin` and `viewer` are set in the database.
- Passwords are stored as salted PBKDF2-SHA256 hashes, never in plain text. Tokens are stored only as SHA-256 hashes, so a database leak does not expose working tokens.
- Login gives the same `401 Invalid email or password` for an unknown email and a wrong password, so it does not reveal which emails have accounts. Accounts created before password login (no password set) cannot log in.
- No cookies are used: the token is returned in the JSON body. CORS already allows the `Authorization` header from any origin.
- The other endpoints (1 to 15) do **not** require a token yet.

User object

Field	Type	Description
<code>id</code>	string	User id
<code>tenantId</code>	string	Tenant the account belongs to
<code>email</code>	string	Lower-case email
<code>name</code>	string null	Display name
<code>role</code>	string	<code>admin</code> , <code>operator</code> or <code>viewer</code>
<code>createdAt</code>	string	UTC timestamp

Session object (returned by register and login)

Field	Type	Description
<code>token</code>	string	Bearer token (43 characters). Shown only once; it cannot be retrieved again
<code>tokenType</code>	string	Always <code>"Bearer"</code>

Field	Type	Description
expiresAt	string	UTC time the token stops working (7 days after sign-in)
user	object	The user object

16. POST /api/v1/auth/register

CRUD: Create. Creates the account (User) and signs it in (Session).

Body

Field	Type	Required	Rules
email	string	Yes	Valid email, at most 254 characters. Stored in lower case
password	string	Yes	8 to 128 characters
name	string	No	1 to 100 characters

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/auth/register \
-H "content-type: application/json" \
-d '{ "email": "ana@example.com", "password": "Test-pass-2026", "name": "Ana" }'
```

201 Created (session object)

```
{
  "token": "qf-DwwZ1sN8mX0c3Y1A9pKe2RtV7uB4hJ6gW5oLiFdQ",
  "tokenType": "Bearer",
  "expiresAt": "2026-10-10T07:28:41.754Z",
  "user": {
    "id": "user_6aff46d70f004d29",
    "tenantId": "tenant_moncha_internal",
    "email": "ana@example.com",
    "name": "Ana",
    "role": "operator",
    "createdAt": "2026-10-03T07:28:41.632Z"
  }
}
```

Errors

Status	Example
400	fieldErrors.password: ["Use at least 8 characters"]
400	fieldErrors.email: ["Invalid input: expected string, received undefined"] (missing)
400	Invalid JSON body
409	{ "error": { "code": "conflict", "message": "An account with this email already exists" } }

17. POST /api/v1/auth/login

CRUD: Create. Checks the password and creates a new session.

Body

Field	Type	Required	Rules
email	string	Yes	Valid email (any case)
password	string	Yes	1 to 128 characters

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/auth/login \
-H "content-type: application/json" \
-d '{ "email": "ana@example.com", "password": "Test-pass-2026" }'
```

200 OK (session object, same shape as register)

```
{
  "token": "24b3TgV9cK1mQ7zR2xLp8sW4yN6aE0dJ3hU5oFiBtGk",
  "tokenType": "Bearer",
  "expiresAt": "2026-10-10T07:28:46.854Z",
  "user": {
    "id": "user_6aff46d70f004d29",
    "tenantId": "tenant_moncha_internal",
    "email": "ana@example.com",
    "name": "Ana",
    "role": "operator",
    "createdAt": "2026-10-03T07:28:41.632Z"
  }
}
```

Errors

Status	Example
400	<code>fieldErrors.password: ["Invalid input: expected string, received undefined"]</code>
401	<code>{ "error": { "code": "unauthorized", "message": "Invalid email or password" } }</code>

18. POST /api/v1/auth/logout

CRUD: Update. Marks the session as revoked (revokedAt); the token stops working at once. No body.

Header	Required
authorization: Bearer <token>	Yes

```
curl -X POST https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/auth/logout \
-H "authorization: Bearer 24b3TgV9cK1mQ7zR2xLp8sW4yN6aE0dJ3hU5oFiBtGk"
```

200 OK

```
{ "ok": true }
```

Logout is safe to repeat: an expired, unknown or already logged-out token also returns `200 { "ok": true }`.

401 when the header is missing:

```
{ "error": { "code": "unauthorized", "message": "Missing Authorization: Bearer <token> header" } }
```

19. GET /api/v1/auth/me

CRUD: Read. Returns the user of the token.

Header	Required
authorization: Bearer <token>	Yes

```
curl https://moncha-backend.vinothjv4-tech.workers.dev/api/v1/auth/me \
-H "authorization: Bearer 24b3TgV9cK1mQ7zR2xLp8sW4yN6aE0dJ3hU5oFiBtGk"
```

200 OK

```
{
  "user": {
    "id": "user_6aff46d70f004d29",
    "tenantId": "tenant_moncha_internal",
    "email": "ana@example.com",
    "name": "Ana",
    "role": "operator",
    "createdAt": "2026-10-03T07:28:41.632Z"
  }
}
```

401

When	message
No Authorization header	Missing Authorization: Bearer <token> header
Token unknown, expired or logged out	Session expired or logged out

Frontend example

```
const API = 'https://moncha-backend.vinothjv4-tech.workers.dev';

// Login
const res = await fetch(`${API}/api/v1/auth/login`, {
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
if (!res.ok) throw new Error((await res.json()).error.message);
const { token, user } = await res.json();
localStorage.setItem('moncha_token', token);

// Current user (on page load)
const me = await fetch(`${API}/api/v1/auth/me`, {
  headers: { authorization: `Bearer ${localStorage.getItem('moncha_token')}` },
});
if (me.status === 401) { /* go to login */ }

// Logout
await fetch(`${API}/api/v1/auth/logout`, {
  method: 'POST',
  headers: { authorization: `Bearer ${localStorage.getItem('moncha_token')}` },
});
localStorage.removeItem('moncha_token');
```

How a lead's queue is decided

The worker audits the lead's website in up to three passes and stops at the first clear answer:

1. **HTML check:** fetches the page and looks for known chatbot and live-chat scripts.
2. **Browser render (Playwright):** opens the site in a real browser (home page plus a few internal pages) when the HTML check is not sure. Skipped when the site's `robots.txt` blocks crawlers.
3. **AI check (OpenRouter):** only when the first two are still not sure. The AI's answer, confidence and reasons are stored on the audit.

The result then sets `queue` and `qualificationReason`:

Result	queue	qualificationReason
Company has no website	NO_WEBSITE	no_website
Audit not finished yet	PENDING_AUDIT	pending_audit
Site is down, parked or blocks access	INACTIVE	website_inactive , website_parked , website_inaccessible
Chatbot or live chat found	HAS_ASSISTANT	has_assistant
No chatbot, confidence ≥ 0.8 (HTML, render, or AI with confidence ≥ 0.85)	QUALIFIED	no_assistant_high_confidence
No chatbot, but confidence below 0.8	NEEDS_REVIEW	no_assistant_below_confidence
Still not sure after all passes	NEEDS_REVIEW	uncertain

WhatsApp, Messenger, Telegram, LINE and Viber links are contact channels, not chatbots, so a site with only those can still be `QUALIFIED`.

Enums

Enum	Values
Lead queue	<code>PENDING_AUDIT</code> , <code>QUALIFIED</code> , <code>HAS_ASSISTANT</code> , <code>NO_WEBSITE</code> , <code>NEEDS_REVIEW</code> , <code>INACTIVE</code>
Assistant verdict	<code>NO_ASSISTANT</code> , <code>HAS_ASSISTANT</code> , <code>UNCERTAIN</code> , <code>NOT_APPLICABLE</code>
Website status	<code>UNCHECKED</code> , <code>ACTIVE</code> , <code>INACTIVE</code> , <code>PARKED</code> , <code>INACCESSIBLE</code> , <code>MISSING</code>
Job status	<code>pending</code> , <code>running</code> , <code>done</code> , <code>failed</code>
Job type	<code>country_discovery</code> , <code>website_audit</code> , <code>places_discovery</code> , <code>csv_import</code>
Schedule run trigger	<code>schedule</code> , <code>manual</code>
User role	<code>admin</code> , <code>operator</code> , <code>viewer</code>

Background processing

The API only stores requests. The work is done by the background worker (`apps/worker/src/poller.ts`):

API call	Needs the worker to
<code>POST /api/v1/schedules</code>	Start the crawl when the time is due
<code>POST /api/v1/discovery/country</code>	Run the crawl (job goes <code>pending</code> → <code>running</code> → <code>done</code>)
<code>POST /api/v1/leads</code> with a domain	Audit the website (lead leaves <code>PENDING_AUDIT</code>)
<code>POST /api/v1/leads/:id/audit</code>	Run the re-audit

On the deployed backend the worker container is not running (the Cloudflare account has no Containers access), so these jobs stay `pending` until a worker runs, for example `pnpm --filter @moncha/worker start` locally. Poll `GET /api/v1/jobs/:id` until `status` is `done` or `failed`.

Local API differences

The local API (`pnpm --filter @moncha/worker api` on port 4000, or the worker itself on `HEALTH_PORT`) serves the same endpoints 2 to 19 with the same payloads. Differences from the deployed backend:

Area	Local API	Deployed backend
<code>GET /</code>	Not served (404)	Service index

Area	Local API	Deployed backend
Wrong method on a known path	405 <code>method_not_allowed</code>	404 <code>not_found</code>
Body over 1 MB	413 <code>payload_too_large</code>	No 1 MB check (Cloudflare's own request size limit applies)
No tenant	400 if no <code>x-tenant-id</code> and no <code>DEFAULT_TENANT_ID</code>	<code>DEFAULT_TENANT_ID</code> , else <code>tenant_moncha_internal</code>
<code>GET /health</code> when run by the worker	Adds <code>"worker": { "workerId", "startedAt", "crawlRunning" }</code>	No <code>worker</code> field
Lead <code>company.website</code>	All website fields filled	Only <code>id</code> , <code>url</code> , <code>status</code> , <code>title</code>
Lead <code>company.sourceRecords</code>	Filled	Always <code>[]</code>
<code>openReviewTasks</code> in counts	Real count	Always <code>0</code>

Test report

Auth (2026-10-03, deployed version `a67dc3fb` , and the local API): 22 passed, 0 failed on each. Register: empty body 400, short password 400, bad email 400, invalid JSON 400, ok 201, duplicate email in other case 409. Login: missing password 400, wrong password 401, unknown email 401, account without password 401, ok with email in other case 200. Me: ok 200, no token 401, bogus token 401, after logout 401, other session still valid 200. Logout: no token 401, ok 200, repeat 200, second session 200. Unknown auth route 404, wrong method 404 (local 405).

Re-check (2026-10-03, after the audit changes): 26 passed, 0 failed. Read-only and validation cases only, so no test data was written: index, health, CORS preflight 204, schedules list, runs, runs `limit=500` 400, bad time 400, unsupported country 400, invalid JSON 400, delete without country 400, delete unknown schedule 404, discovery missing country 400, `maxPages=4` 400, unknown job 404, get job 200, leads list 200, bad queue 400, `pageSize=101` 400, filtered list 200, counts 200, get lead 200, unknown lead 404, create without name 400, re-audit unknown lead 404, unknown path 404, `PUT` 404.

Full run on 2026-10-03 against the deployed backend (version `4985559e`): **50 passed, 0 failed.**

Area	Cases
Service	index 200, health 200, CORS preflight 204, unknown path 404, unknown API path 404, wrong method 404
Schedules	list 200; create 201 (default zone and <code>Asia/Tokyo</code> , by name and by code); duplicate 409; bad time 400; unsupported country 400; bad time zone 400; invalid JSON 400; delete by id 200 then 404; delete by country 200; missing country 400; unsupported country 400; runs 200; <code>limit=2</code> 200; <code>limit=500</code> 400
Discovery and jobs	missing country 400; unsupported country 400; <code>maxPages=4</code> 400; create 202; get job 200; unknown job 404

Area	Cases
Leads	list default 200; queue + page 200; search 200; country 200; bad queue 400; pageSize=101 400; counts 200; get 200; unknown 404; create missing name 400; create duplicate 200; create without website 201; create with website 201; same domain as URL 200 duplicate; re-audit with open audit 202 deduped; re-audit without domain 404; re-audit unknown lead 404; re-audit new job 202; re-audit again 202 deduped